

ZERO TO PERSONAL AI EMPLOYEE · 6 SCENARIOS

OpenClaw with General Agents

Hand your agent a tiny brief. Walk away with an AI employee that lives on your laptop and answers from your phone.

● ● ● ~/OPENCLAW

```
$ claude # your agent reads the brief and gets to work
```

THE SHIFT

The way we work just changed.

We no longer click through software. We **describe what we want** and a general agent does the work — installing, configuring, debugging, verifying.

Claude Code, OpenCode, Cowork — the interface is now a conversation. The question is no longer *"which app?"*
It's *"what should I delegate?"*

WHAT WE'RE BUILDING

Meet the Personal AI Employee.

Not a chatbot you visit. A worker you **delegate to** — running on your own laptop, replying through the apps you already use.

01

Lives on your laptop

Open-source, runs locally. Your data stays with you — not a vendor's cloud.

02

Answers from your phone

WhatsApp, Telegram, Discord, Slack, iMessage. Message it like a colleague.

03

Works on its own

Uses tools, customizes to you, and runs scheduled tasks without you asking.

WHY IT MATTERS

It already proved AI employees are real.

100k+

GitHub stars in its first months

“The next ChatGPT.”

Jensen Huang, GTC 2026 –
NVIDIA went on to build
NemoClaw on top of it.

The fastest-growing open-source
project of 2026.

THE CATCH

So why doesn't everyone have one?

Setup is the wall. Keys, services, config files, background processes, channel pairing, debugging when it breaks.

Powerful tools die on the install step. The people who'd benefit most — students, knowledge workers — never make it past the terminal.

THE WHOLE IDEA

Don't install it. Delegate the install.

You don't learn the commands. Your general agent does the setup — install, configure, debug, restart, verify — and you approve along the way.

WHAT YOU HAND THE AGENT

The whole “skill” is two files.

AGENTS.md

~600 lines

An operational reference for any general agent doing OpenClaw work — principles, operations, gotchas, safety rails.

+

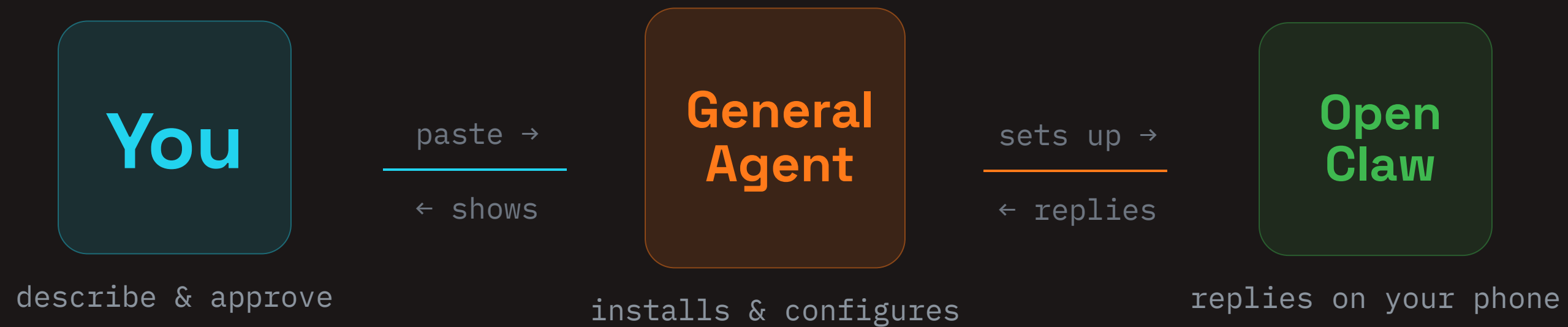
CLAUDE.md

1 line

Just **@AGENTS.md** — tells your agent to import the brief automatically the moment a session starts.

One file plus a one-line index *is* the entire environment. Download it once; nothing in it goes stale.

Three actors share one page.



You steer. The agent works. OpenClaw becomes the employee that answers and runs tasks for you.

EVERY SCENARIO, THE SAME SHAPE

One rhythm, five steps.

- | | | |
|-----------|-----------------|---|
| 01 | Describe | You paste one sentence — a brief, not a script. |
| 02 | Plan | The agent consults the brief and proposes a plan, flagging decision points. |
| 03 | Approve | You approve and watch it run, restart, and verify. |
| 04 | Seam | It stops where only you can act — a key, a QR scan, an OAuth screen. |
| 05 | Done | You're finished when one observable thing happens. Each scenario names it. |

THE PATH

Six scenarios. Ninety minutes. One employee.

01 • 15 min

Install & chat

Running locally, replying on Gemini's free tier.

02 • 15 min

Pair your phone

Message it from WhatsApp, Telegram, or Discord.

03 • 10 min

Delegate real work

Watch the agent loop run on a real task.

04 • 15 min

Sound like you

Voice, identity, memory — then back it up.

05 • 15 min

Extend it

Add one skill and one external tool.

06 • 15 min

Act on its own

One scheduled task that runs without you.

BEFORE YOU START

Three things on your machine.

1 A general agent installed

Claude Code or OpenCode — either works. Every paste prompt is identical across both.

2 Comfort approving agents

You can approve a tool call, read agent output, and tell when it's stuck.

3 Node.js 22.16 or later

Run `node --version`. Below that, your agent will walk you through the upgrade.

SCENARIO 01 • 15 MIN

Get it installed and chatting.

OpenClaw running locally, Gemini on the free tier, and a real reply when you type “hi”.

01

Describe the goal. Approve the plan.

●●● PASTE TO YOUR AGENT

```
> I'd like OpenClaw running on my laptop and chatting back through Gemini's free tier.
Before you touch anything, walk me through your plan — what you'll check, what you'll
change, and where you need me.
```

It reads the brief, looks at your machine, and proposes a plan. It pauses twice for you: to grab a free Gemini key from aistudio.google.com, and to confirm before any change.

DONE WHEN

Your agent's own CLI check replies, *and* the dashboard replies after you type hi — footer showing google/gemini-2.5-flash.

UNDER THE HOOD

What's actually running on your laptop.



A background service starts when you log in. The **gateway** is the always-on substrate everything else plugs into.

SCENARIO 02 • 15 MIN

Pair your phone.

Send “hi” from your phone to your AI employee — and get a reply back.

02

The one step only you can cross.

The agent sets up the channel — but the login uses a small terminal UI for the QR code that won't render through its shell.

So it pauses and asks you to run the login command in a fresh terminal. You scan the QR from your phone, or paste a bot token. Then tell it “`linked.`”

THE SEAM

```
# agent stops here
> Open a fresh terminal and run
  openclaw channel login whatsapp

# you scan the QR → reply:
> linked
```

DONE WHEN

You send `hi` from your phone to the bound number and a real reply comes back.

SCENARIO 03 • 10 MIN

Delegate real work.

Send a task that needs the agent to actually go and do something — and watch the loop.

03

Six lines scroll past. That's the loop.

● ● ● GATEWAY LOG · LIVE

```
1 inbound message arrives on your channel
2 model loop asks Gemini what to do
3 tool agent calls web fetch / file / calendar
4 result the tool returns content
5 model loop sends it back to summarize
6 outbound reply returns to your channel
```

Pick a real task — the kind ChatGPT would refuse or do poorly. It needs live data, reasoning, and structure.

Once you've watched the loop run, you can name what your employee does *every* time it replies. Later scenarios just add tools and triggers inside this same loop.

DONE WHEN

You've seen that six-line shape scroll past and the reply lands on your phone.

SCENARIO 04 • 15 MIN

Sound like you. Remember you.

Give it your voice, a name, context about you, and memory that crosses channels.

04

SCENARIO 04 · THE BRAIN

Its behavior lives in markdown files.

Edit four files in `~/.openclaw/workspace/`. Send `/reset` to reload. Send one message. Feel the difference.

SOUL.md

Personality and tone — *how it talks.*

IDENTITY.md

Its name and role — *how it introduces itself.*

USER.md

What it knows about you — *the persistent context.*

MEMORY.md

Durable facts — *committed across channels.*

Keep each lean — every line is context the agent pays on every single reply. A page or two each is plenty.

Three layers of memory.

Session memory

in RAM

Automatic. Survives only until `/reset`.

Channel memory

on disk, per channel

Survives restarts — but *per channel*. The dashboard doesn't see what WhatsApp said.

Long-term memory

MEMORY.md

The only layer that crosses channels — and only with a **deliberate commit**.

DONE WHEN

A fact you commit in WhatsApp is recalled in the dashboard. The commit crossed the wall.

Back up the identity you built.

The workspace *is* your AI employee — the brain files, the tool policy, the schedules. If your laptop dies tonight, you lose all of it. Treat it like dotfiles.

● ● ● PASTE TO YOUR AGENT

```
> Back up my workspace to a private GitHub repo.  
Exclude secrets and caches. Give me a one-liner that  
re-clones it onto a fresh laptop.
```

DONE WHEN

The private repo exists, the workspace is pushed, and a recovery one-liner is saved somewhere you'll find it.

SCENARIO 05 • 15 MIN

Extend it.

Add one skill and one external tool — and learn the pattern every extension follows.

05

Skills add know-how. Tools add reach.

A SKILL

Expertise it auto-invokes

A folder with a `SKILL.md`. The agent reaches for it when a task matches. One cross-runtime spec works in OpenClaw, Claude Code, and 50+ others.

`install · restart · verify · test`

AN MCP TOOL

Capability it can call

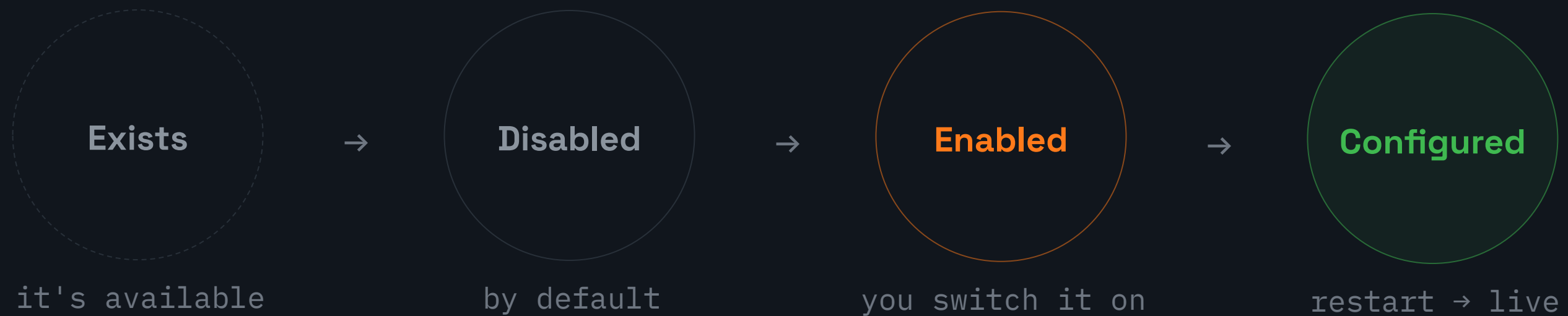
An external service exposing functions over the Model Context Protocol — query a database, send a calendar invite, get the time anywhere. No code.

`configure · restart · verify · test`

Watch for silent failures: a skill that doesn't fire is usually a description mismatch; a missing MCP tool shows up only in the gateway log.

THE PATTERN BEHIND EVERY EXTENSION

The activation dance.



Skills, plugins, MCP servers, channels, hooks — all four steps, every time. When a feature feels broken on first try, walk the dance.

SCENARIO 06 • 15 MIN

Make it act on its own.

One schedule that fires without you messaging it. This is the proactive dimension.

06

Cron, heartbeat, hooks.

precise times

Cron

“Every morning at 7am.” “Every Monday at 9.” What you'll use most — your days run on clock times.

ambient cadence

Heartbeat

“Every 30 minutes, scan for urgent unread.” For checking on something periodically.

event triggers

Hooks

A webhook fires, a session resets. Out of scope today — the advanced path.

DONE WHEN

One real schedule is running, committed to your backup repo, and your agent has told you when it next fires. Leave it on.

THE HABIT · 10 MIN A MONTH

The monthly audit.

Your employee accumulates — skills, credentials, tools, memory, autonomous tool calls. Each was a small decision you approved. The chain compounds quietly.

The defense isn't vigilance at install time. It's a ten-minute review on a fixed cadence.

● ● ● PASTE TO YOUR AGENT · MONTHLY

```
> Run my monthly audit. Walk through everything installed, stored, scheduled, or written since last time. Flag anything I didn't approve. Summarize as one short report.
```

DONE WHEN

You've reviewed the report and made at least one decision — trim a credential, revoke an approval, prune a stale memory.

OPENCLAW

AFTER SIX SCENARIOS

Not six demos. One system.

Background service	Survives closing the terminal and rebooting.
Channel pairing	The trusted path your phone uses to reach it.
Workspace + GitHub backup	Its identity, context, memory — surviving a laptop wipe.
One skill • one tool • one schedule	Real know-how, real reach, and one thing that runs for you.

None of these are demos you disabled. All of them are pieces of a tool you'll touch tomorrow.

WHAT IT FEELS LIKE

A working day, hands-free.

7am Your phone buzzes with the schedule you chose — today's summary.

11am A quick question triggers the time MCP or the skill you installed.

3pm You ask it to draft replies to three emails. It does.

6pm You commit one new fact to long-term memory.

You never opened your laptop.

WHY THIS HOLDS UP

Two things stay fresh. One stays durable.

fresh

The scenarios

Live on the book site. Your agent fetches the current version every session.

fresh

The commands

OpenClaw ships fast. The agent reads `llms.txt` fresh before any command it's unsure of.

durable

AGENTS.md

What OpenClaw is, how to navigate it, the safety rails. Download once; never goes stale.

The intelligence isn't in the files. It's in your agent reading them and applying them to whatever you ask next.

THE META-LESSON

Now write the **next one.**

Every tool with a learnable surface has a little brief worth writing. OpenClaw was the early example. You'll find others — author them, and hand your agent the keys.

 YOU ASSEMBLED A TOOL YOU'LL TOUCH TOMORROW

```
$ # six scenarios. zero to Personal AI  
Employee.
```